Knowledge Engineering and Data Science

Programme: B.E./B.Tech. (Undergraduate) |

Course: Knowledge Engineering and Data Science

Experiment 1: Data Exploration and Cleaning with pandas

Level: Simple | **CO:** CO3 | **Duration:** 2 hours | **Marks:** 8

Aim: To load a dataset, inspect it, handle missing and inconsistent values, and summarise it using descriptive statistics and charts.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
# 1. Load dataset
df = pd.read_csv('student_marks.csv')
# 2. Explore dataset
print(df.head())
print(df.shape)
df.info()
print(df.describe())
# 3. Check missing values
print("\nMissing Values:")
print(df.isna().sum())
# 4. Fill missing values
num = ['attendance', 'internal', 'final_exam', 'study_hours']
cat = ['department', 'gender']
for col in num:
    df[col] = df[col].fillna(df[col].median())
for col in cat:
    df[col] = df[col].fillna(df[col].mode()[0])
# 5. Remove duplicates
df = df.drop_duplicates()
# 6. Correct invalid attendance
df.loc[df['attendance'] > 100, 'attendance'] = df.loc[
    df['attendance'] <= 100, 'attendance'
].median()
print("\nCleaned Data:")
print(df)
# 7. Summary statistics
print("\nFinal Exam Statistics:")
```

```
print(df['final_exam'].agg(['mean', 'median', 'std', 'min', 'max']))

# 8. Department-wise average
print("\nDepartment-wise Marks:")
print(df.groupby('department')['final_exam'].mean())
# 9. Histogram
plt.hist(df['final_exam'], bins=5, color='skyblue', edgecolor='black')
plt.title('Histogram of Final Exam Marks')
plt.xlabel('Final Exam Marks')
plt.ylabel('Students')
plt.show()
# 10. Box plot
df.boxplot(column='final_exam', by='department')
plt.title('Marks by Department')
plt.suptitle('')
plt.xlabel('Department')
plt.ylabel('Final Exam Marks')
plt.show()
# 11. Scatter plot
plt.scatter(df['study_hours'], df['final_exam'], color='green')
plt.title('Study Hours vs Final Exam Marks')
plt.xlabel('Study Hours')
plt.ylabel('Final Exam Marks')
plt.show()
# 12. Correlation
print("\nCorrelation:")
print(df['study_hours'].corr(df['final_exam']))
# 13. Save cleaned dataset
df.to_csv('student_marks_clean.csv', index=False)
print("\nCleaned file saved successfully!")
# Download file
from google.colab import files
files.download('student_marks_clean.csv')
```

Output:

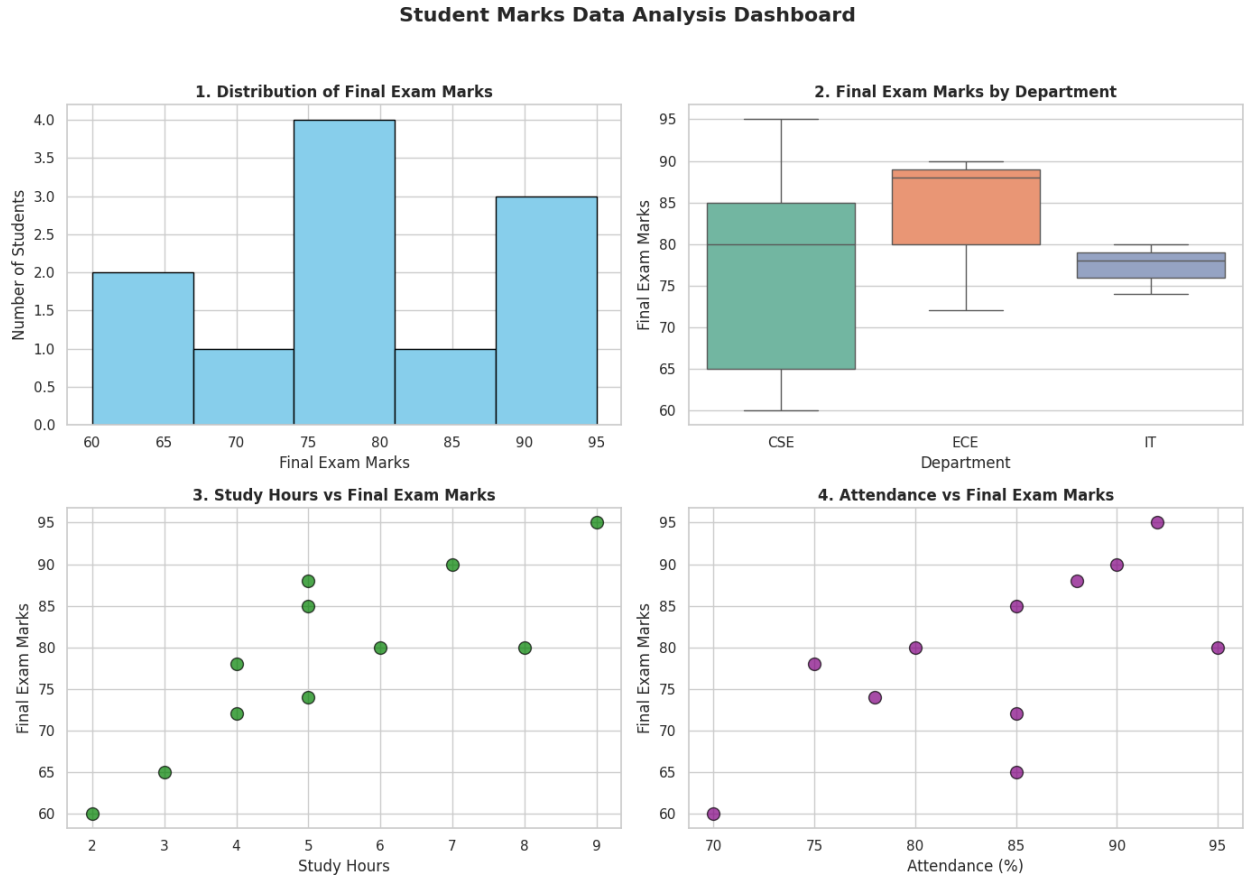
```
1. DATASET OVERVIEW
- Total Records: 11
- Total Columns: 7

2. DEMOGRAPHICS & DEPARTMENTS
- Student Count by Department:
  * CSE: 5 students
  * ECE: 3 students
  * IT: 3 students
- Gender Distribution:
  * Male: 6 students
  * Female: 5 students

3. FINAL EXAM MARKS STATISTICS
- Average (Mean) Score: 78.82
- Median Score: 80.00
- Standard Deviation: 10.62
- Minimum Score: 60.00
- Maximum Score: 95.00

4. DEPARTMENT-WISE PERFORMANCE (AVERAGE MARKS)
- CSE: 77.00
- ECE: 83.33
- IT: 77.33

5. CORRELATION ANALYSIS
- Correlation between Study Hours and Final Exam Marks: 0.8223
==========================================
```



Experiment 2: Semantic Network and Inheritance Queries

Level: Simple | **CO:** CO1 | **Duration:** 2 hours | **Marks:** 8

Aim: To represent a domain as a semantic network and to answer questions by inheritance and path search.

Code:

```
import networkx as nx
import matplotlib.pyplot as plt
# 1. Create a directed graph
G = nx.DiGraph()
# 2. Add nodes and relationships
G.add_edge('Animal', 'Bird', relation='is-a')
G.add_edge('Animal', 'Fish', relation='is-a')
G.add_edge('Bird', 'Canary', relation='is-a')
G.add_edge('Bird', 'Penguin', relation='is-a')
G.add_edge('Canary', 'Tweety', relation='instance-of')
# Add properties
G.add_edge('Animal', 'has life', relation='can')
G.add_edge('Bird', 'wings', relation='has-part')
G.add_edge('Bird', 'fly', relation='can')
G.add_edge('Fish', 'swim', relation='can')
G.add_edge('Canary', 'yellow', relation='colour')
G.add_edge('Penguin', 'cannot fly', relation='can')
# 3. Draw the semantic network
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True,
        node_color='lightblue',
        node_size=2500,
        font_size=9,
        arrows=True)
labels = nx.get_edge_attributes(G, 'relation')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
plt.title("Semantic Network of Animals")
plt.show()
# 4. Function for inheritance
def inherits(node, relation):
    current = node
    path = [node]
    while True:
        # Check properties of current node
        for neighbor in G.successors(current):
            if G[current][neighbor]['relation'] == relation:
                return neighbor, path
```

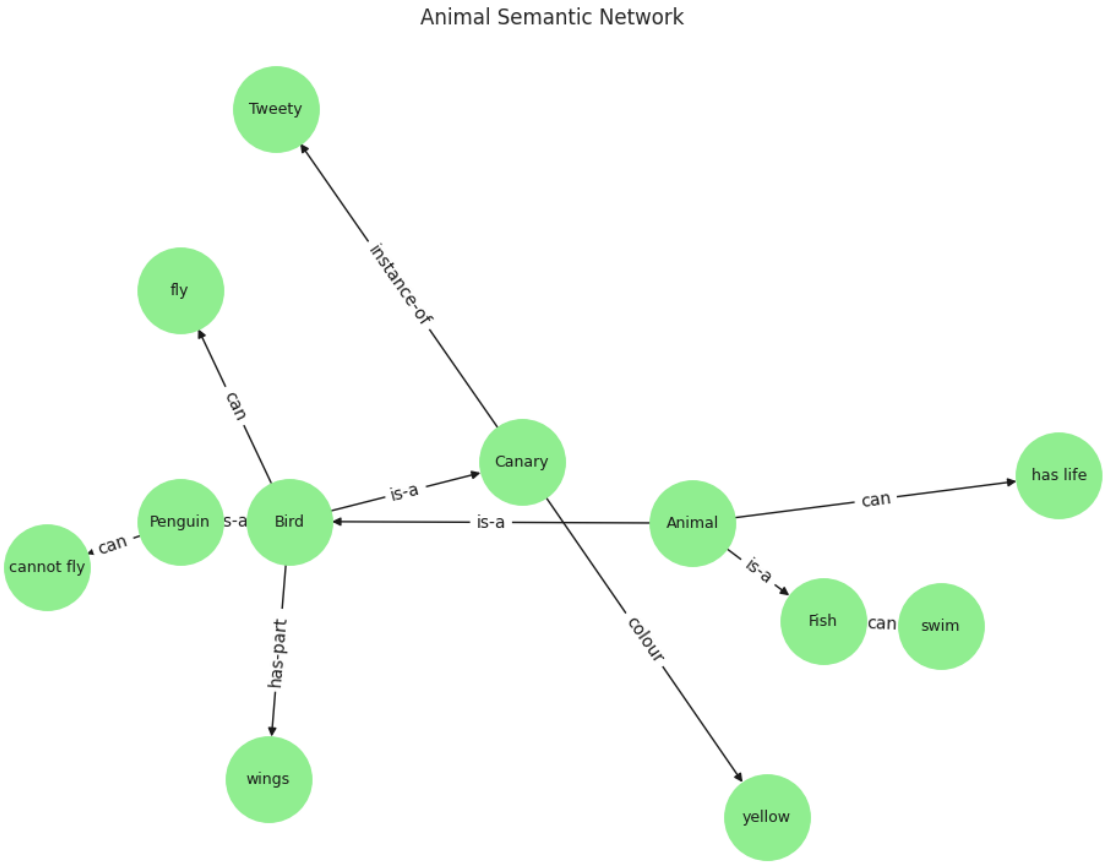
```
        # Find parent using is-a or instance-of
        parents = [
            n for n in G.predecessors(current)
            if G[n][current]['relation'] in ['is-a', 'instance-of']
        ]
        if not parents:
            return None, path
        current = parents[0]
        path.append(current)
# 5. Test inheritance queries
queries = [
    ('Tweety', 'can'),
    ('Tweety', 'has-part'),
    ('Tweety', 'colour'),
    ('Penguin', 'can'),
    ('Fish', 'can')
]
print("\nINHERITANCE QUERIES")
for node, relation in queries:
    result, path = inherits(node, relation)
    print("\nQuestion:", node, relation)
    if result:
        print("Answer:", result)
    else:
        print("Answer: Not found")
    print("Path:", " -> ".join(path))
# 6. Find path between two concepts
def find_path(start, end):
    try:
        path = nx.shortest_path(G, start, end)
        print("\nPath from", start, "to", end, ":")
        print(" -> ".join(path))
    except nx.NetworkXNoPath:
        print("No path found")
find_path('Tweety', 'Animal')
# 7. Save the network drawing
plt.figure(figsize=(10, 7))
nx.draw(G, pos, with_labels=True,
        node_color='lightgreen',
        node_size=2500,
        font_size=9,
        arrows=True)
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
```

```
plt.title("Animal Semantic Network")
plt.savefig('semantic_network.png')
plt.show()
print("\nSemantic Network Experiment Completed!")
```

Expected output: Question: Tweety colour
Answer: yellow
Path: Tweety -> Canary

Question: Penguin can
Answer: cannot fly
Path: Penguin

Question: Fish can
Answer: swim
Path: Fish
No path found



Semantic Network Experiment Completed

Experiment 3: Frames with Defaults, Inheritance and If-Needed Procedures

Level: Simple | **CO:** CO1 | **Duration:** 2 hours | **Marks:** 8

Aim: To implement frames in Python with slots, default values, inheritance and attached procedures.

Code:

```
class Frame:
    def __init__(self, name, parent=None):
        self.name = name
        self.parent = parent
        self.slots = {}
    def add_slot(self, slot, value=None, default=None, if_needed=None, if_added=None):
        self.slots[slot] = {
            'value': value,
            'default': default,
            'if_needed': if_needed,
            'if_added': if_added
        }
    def get(self, slot):
        if slot in self.slots:
            data = self.slots[slot]
            if data['value'] is not None:
                return data['value'], 'Frame'
            if data['if_needed']:
                return data['if_needed'](), 'Procedure'
            if data['default'] is not None:
                return data['default'], 'Default'
        if self.parent:
            return self.parent.get(slot)
        return None, 'Not Found'
    def set(self, slot, value):
        if slot not in self.slots:
            self.add_slot(slot)
        self.slots[slot]['value'] = value
        demon = self.slots[slot]['if_added']
        if demon:
            demon(value)
def check_mileage(value):
    if value < 0:
        print("Warning: Mileage cannot be negative!")
def calculate_mileage():
    return 12000 / 600
```

```
# Create frames
Vehicle = Frame('Vehicle')
Vehicle.add_slot('wheels', default=4)
Vehicle.add_slot('mileage', if_needed=calculate_mileage)
Car = Frame('Car', Vehicle)
Car.add_slot('fuel_type', default='Petrol')
Car.add_slot('brand', default='Unknown')
MyCar = Frame('MyCar', Car)
MyCar.add_slot('brand', value='Toyota')
MyCar.add_slot('colour', value='White')
MyCar.add_slot('odometer', value=12000)
MyCar.add_slot('fuel_used', value=600)
MyCar.add_slot('test_mileage', value=100, if_added=check_mileage)
# Display slot values
print("MY CAR FRAME DETAILS")
print("------------------------------")
print("Slot          Value       Source")
print("------------------------------")
slots = ['wheels', 'mileage', 'fuel_type', 'brand', 'colour', 'odometer', 'fuel_used']
for slot in slots:
    value, source = MyCar.get(slot)
    print(f"{slot:<14}{str(value):<12}{source}")
# Demonstrate demon
print("\nDEMON DEMONSTRATION")
MyCar.set('test_mileage', -50)
print("\nExperiment Completed!")
```

Expected output:

```
Y CAR FRAME DETAILS
------------------------------
Slot          Value       Source
------------------------------
wheels        4           Default
mileage       20.0        Procedure
fuel_type     Petrol      Default
brand         Toyota      Frame
colour        White       Frame
odometer      12000       Frame
fuel_used     600         Frame

DEMON DEMONSTRATION
Warning: Mileage cannot be negative!

Experiment Completed!
```

Experiment 4: Rule-Based System with Forward Chaining

Level: Simple | **CO:** CO2 | **Duration:** 2 hours | **Marks:** 8

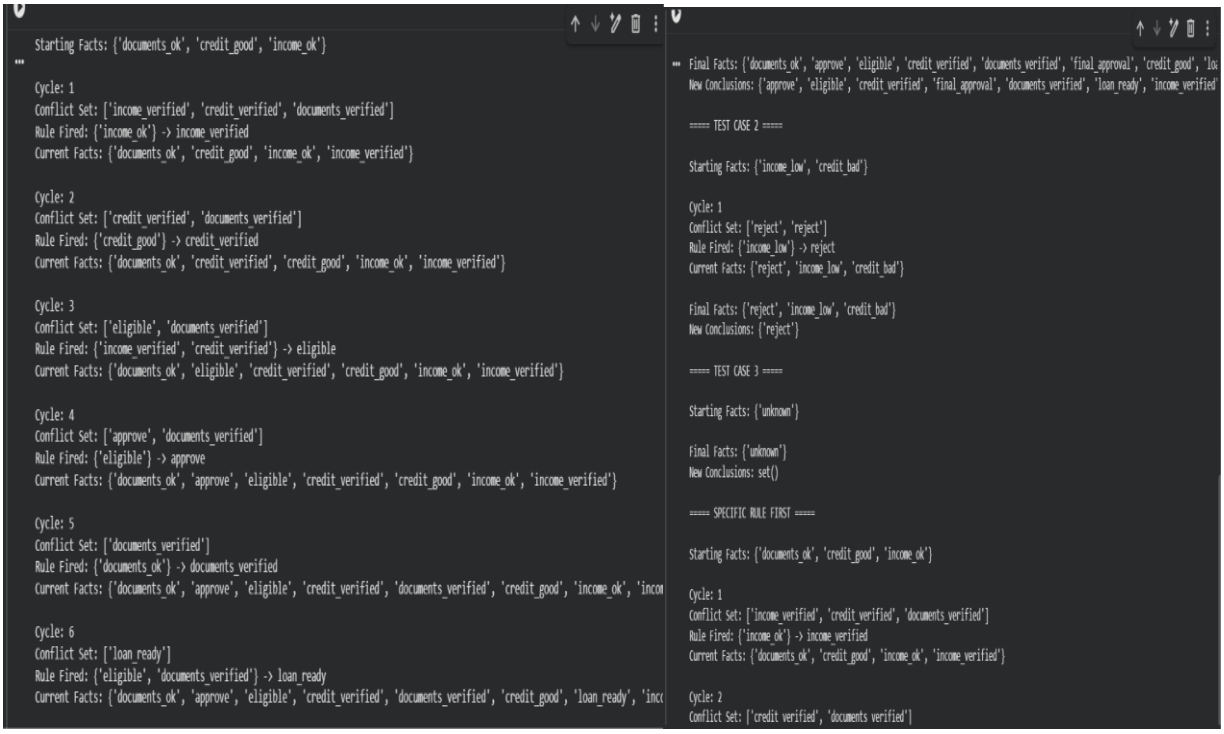
Aim: To build a small production system that applies IF-THEN rules to a set of facts by forward chaining.

Code:

```
# Experiment 4: Forward Chaining
rules = [
    ({"income_ok"}, "income_verified"),
    ({"credit_good"}, "credit_verified"),
    ({"income_verified", "credit_verified"}, "eligible"),
    ({"eligible"}, "approve"),
    ({"income_low"}, "reject"),
    ({"documents_ok"}, "documents_verified"),
    ({"eligible", "documents_verified"}, "loan_ready"),
    ({"loan_ready"}, "final_approval"),
    ({"credit_bad"}, "reject")
]
def forward_chaining(facts, specific=False):
    facts = set(facts)
    initial = facts.copy()
    cycle = 1
    print("\nStarting Facts:", facts)
    while True:
        matched = [(c, r) for c, r in rules
                   if c.issubset(facts) and r not in facts]
        if not matched:
            break
        if specific:
            matched.sort(key=lambda x: len(x[0]), reverse=True)
        print("\nCycle:", cycle)
        print("Conflict Set:", [r for c, r in matched])
        conditions, conclusion = matched[0]
        print("Rule Fired:", conditions, "->", conclusion)
        facts.add(conclusion)
        print("Current Facts:", facts)
        cycle += 1
    print("\nFinal Facts:", facts)
    print("New Conclusions:", facts - initial)
    return facts
# Test Case 1
print("\n===== TEST CASE 1 =====")
starting_facts = {"income_ok", "credit_good", "documents_ok"}
```

```
forward_chaining(starting_facts)
# Test Case 2
print("\n===== TEST CASE 2 =====")
starting_facts = {"income_low", "credit_bad"}
forward_chaining(starting_facts)
# Test Case 3
print("\n===== TEST CASE 3 =====")
starting_facts = {"unknown"}
forward_chaining(starting_facts)
# Most specific rule first
print("\n===== SPECIFIC RULE FIRST =====")
starting_facts = {"income_ok", "credit_good", "documents_ok"}
forward_chaining(starting_facts, specific=True)
print("\nExperiment Completed!")
```

Expected output:



PART B – MEDIUM EXPERIMENTS

Experiment 5: Backward Chaining with a Why/How Explanation Facility

Level: Medium | **CO:** CO2 | **Duration:** 3 hours | **Marks:** 10

Aim: To implement goal-driven reasoning that asks the user for missing facts and can explain why it asks and how it reached a conclusion.

Code: # Experiment 5: Backward Chaining with Why and How

```
# 1. Define rules
rules = [
    ("R1", ["yellow_leaves", "wet_soil"], "root_rot"),
    ("R2", ["brown_roots"], "root_rot"),
    ("R3", ["leaf_spots", "high_humidity"], "fungal_infection"),
    ("R4", ["white_powder", "high_humidity"], "fungal_infection"),
    ("R5", ["yellow_leaves", "dry_soil"], "water_deficiency"),
    ("R6", ["wilting", "dry_soil"], "water_deficiency"),
    ("R7", ["holes_in_leaves", "insects_seen"], "pest_attack"),
    ("R8", ["damaged_stems", "insects_seen"], "pest_attack"),
    ("R9", ["root_rot"], "disease_detected"),
    ("R10", ["fungal_infection"], "disease_detected"),
    ("R11", ["water_deficiency"], "treatment_needed"),
    ("R12", ["pest_attack"], "treatment_needed")
]
# 2. Askable facts
askable = [
    "yellow_leaves", "wet_soil", "brown_roots",
    "leaf_spots", "high_humidity", "white_powder",
    "dry_soil", "wilting", "holes_in_leaves",
    "insects_seen", "damaged_stems"
]
# 3. Backward chaining function
def consultation(goal):
    facts = {}
    trace = []
    questions = []
    def prove(goal, reason="Main goal"):
        if goal in facts:
            return facts[goal]
        matching = [r for r in rules if r[2] == goal]
        if matching:
            for rid, conditions, conclusion in matching:
                print("\nTrying", rid, "to prove", goal)
                success = True
                for condition in conditions:
                    if not prove(condition, rid):
                        success = False
                        break
                if success:
                    trace.append((rid, conditions, conclusion))
                    return True
```

```
        if goal in askable:
            while True:
                answer = input(
                    "\nWhy is the system asking about " +
                    goal + "? Type yes/no/why: "
                ).lower().strip()
                if answer == "why":
                    print("WHY: This question is required by",
                          reason, "to prove", goal)
                    continue
                if answer in ["yes", "no"]:
                    facts[goal] = (answer == "yes")
                    questions.append(goal)
                    return facts[goal]
                print("Please enter yes, no or why.")
        return False
    print("\nGOAL:", goal)
    result = prove(goal)
    print("\nHOW EXPLANATION")
    if result:
        for rid, conditions, conclusion in trace:
            print(rid, ":", " AND ".join(conditions),
                  "->", conclusion)
        print("Conclusion proved:", goal)
    else:
        print("Goal could not be proved:", goal)
    print("\nFacts asked:", questions)
    print("Number of questions:", len(questions))
    return result
# 4. Four consultations
print("\n===== CONSULTATION 1 =====")
consultation("root_rot")
print("\n===== CONSULTATION 2 =====")
consultation("fungal_infection")
print("\n===== CONSULTATION 3 =====")
consultation("water_deficiency")
print("\n===== CONSULTATION 4 =====")
consultation("unknown_disease")
# 5. Comparison
print("\n===== COMPARISON =====")
print("Backward Chaining: Asks only required questions.")
print("Forward Chaining: Checks rules against available facts.")
print("\nExperiment Completed!")
```

Expected output: ===== CONSULTATION 1 =====

GOAL: root_rot

Trying R1 to prove root_rot

Why is the system asking about yellow_leaves? Type yes/no/why: yes

Why is the system asking about wet_soil? Type yes/no/why: why
WHY: This question is required by R1 to prove wet_soil

Why is the system asking about wet_soil? Type yes/no/why: no

Trying R2 to prove root_rot

Why is the system asking about brown_roots? Type yes/no/why: yes

HOW EXPLANATION
R2 : brown_roots -> root_rot
Conclusion proved: root_rot

Facts asked: ['yellow_leaves', 'wet_soil', 'brown_roots']
Number of questions: 3

===== CONSULTATION 2 =====

GOAL: fungal_infection

Trying R3 to prove fungal_infection

Why is the system asking about leaf_spots? Type yes/no/why: no

Trying R4 to prove fungal_infection

Why is the system asking about white_powder? Type yes/no/why: no

HOW EXPLANATION
Goal could not be proved: fungal_infection

Facts asked: ['leaf_spots', 'white_powder']
Number of questions: 2

===== CONSULTATION 3 =====

GOAL: water_deficiency

Trying R5 to prove water_deficiency

Why is the system asking about yellow_leaves? Type yes/no/why: yes

Why is the system asking about dry_soil? Type yes/no/why: yes

HOW EXPLANATION
R5 : yellow_leaves AND dry_soil -> water_deficiency
Conclusion proved: water_deficiency

Facts asked: ['yellow_leaves', 'dry_soil']
Number of questions: 2

===== CONSULTATION 4 =====

GOAL: unknown_disease
HOW EXPLANATION
Goal could not be proved: unknown_disease
Facts asked: []
Number of questions: 0
===== COMPARISON =====
Backward Chaining: Asks only required questions.
Forward Chaining: Checks rules against available facts.
Experiment Completed!

Experiment 6: Building and Reasoning over an Ontology

Level: Medium | **CO:** CO1 | **Duration:** 3 hours | **Marks:** 10

Aim: To design a small ontology, create instances, and use a reasoner to check consistency and infer new facts.

Code:

```
from owlready2 import *
# 1. Create ontology
onto = get_ontology("http://example.org/university.owl")

with onto:
    # 2. Create classes
```

```
class Person(Thing): pass
class Student(Person): pass
class Faculty(Person): pass
class Course(Thing): pass
class Department(Thing): pass
class Undergraduate(Student): pass
class Postgraduate(Student): pass
class ResearchScholar(Student): pass
# Student and Faculty are disjoint
AllDisjoint([Student, Faculty])
# 3. Object properties
class enrolledIn(ObjectProperty):
    domain = [Person]
    range = [Course]
class teaches(ObjectProperty):
    domain = [Faculty]
    range = [Course]
class belongsTo(ObjectProperty):
    domain = [Person]
    range = [Department]
# 4. Data properties
class hasName(DataProperty):
    domain = [Person]
    range = [str]
class hasCredits(DataProperty):
    domain = [Course]
    range = [int]
# 5. Course must have at least one teacher
Course.is_a.append(teaches.min(1, Faculty))
# 6. Create individuals
Asha = Student("Asha")
Ravi = Student("Ravi")
Priya = Undergraduate("Priya")
Arun = Postgraduate("Arun")
Kiran = ResearchScholar("Kiran")
Kumar = Faculty("Kumar")
Meena = Faculty("Meena")
Suresh = Faculty("Suresh")
Python = Course("Python")
AI = Course("AI")
DBMS = Course("DBMS")
CSE = Department("CSE")
ECE = Department("ECE")
```

```
    # 7. Assign names
    Asha.hasName = ["Asha"]
    Ravi.hasName = ["Ravi"]
    Kumar.hasName = ["Dr Kumar"]
    Meena.hasName = ["Dr Meena"]
    # 8. Assign course credits
    Python.hasCredits = [4]
    AI.hasCredits = [3]
    DBMS.hasCredits = [4]
    # 9. Add relationships
    Asha.enrolledIn = [Python, AI]
    Ravi.enrolledIn = [DBMS]
    Priya.enrolledIn = [Python]
    Arun.enrolledIn = [AI]
    Kiran.enrolledIn = [DBMS]
    Kumar.teaches = [Python, AI]
    Meena.teaches = [DBMS]
    Suresh.teaches = [Python]
    Asha.belongsTo = [CSE]
    Ravi.belongsTo = [ECE]
    Kumar.belongsTo = [CSE]
    Meena.belongsTo = [ECE]
# 10. Run reasoner
print("RUNNING REASONER")
with onto:
    sync_reasoner(infer_property_values=True)
# 11. Display inferred facts
print("\nINFERRED CLASS MEMBERSHIP")
for person in onto.Person.instances():
    print(person.name, ":", [c.name for c in person.is_a if hasattr(c, 'name')])
# 12. Competency questions
print("\nCOMPETENCY QUESTIONS")
print("\n1. Which courses does Asha attend?")
for course in Asha.enrolledIn:
    print(course.name)
print("\n2. Which courses does Ravi attend?")
for course in Ravi.enrolledIn:
    print(course.name)
print("\n3. Who teaches Python?")
for person in onto.Faculty.instances():
    if Python in person.teaches:
        print(person.name)
print("\n4. Which department does Asha belong to?")
```

```
for dept in Asha.belongsTo:
    print(dept.name)
print("\n5. How many credits does AI have?")
print(AI.hasCredits)
# 13. Create deliberate inconsistency
print("\nCHECKING INCONSISTENCY")
Asha.is_a.append(Faculty)
try:
    with onto:
        sync_reasoner()
    print("No inconsistency detected")
except OwlReadyInconsistentOntologyError:
    print("Inconsistency detected!")
    print("Asha is both Student and Faculty.")
    print("Student and Faculty are disjoint classes.")
# 14. Save ontology
onto.save(file="university.owl", format="rdfxml")
print("\nOntology saved as university.owl")
print("\nEXPERIMENT COMPLETED"
```

Expected output:

```
RUNNING REASONER
* Owlready2 * Running HermiT...
    java -Xmx2000M -cp /usr/local/lib/python3.13/dist-
packages/owlready2/hermit:/usr/local/lib/python3.13/dist-
packages/owlready2/hermit/HermiT.jar org.semanticweb.HermiT.cli.CommandLine -c -O -D -I
file:////tmp/tmpcluunnca -Y
* Owlready2 * HermiT took 2.2137413024902344 seconds
* Owlready * Reparenting university.Course: {owl.Thing} => {university.Faculty}
* Owlready * (NB: only changes on entities loaded in Python are shown, other changes are done
but not listed)
INFERRED CLASS MEMBERSHIP
Asha : ['Student']
Ravi : ['Student']
Priya : ['Undergraduate']
Arun : ['Postgraduate']
Kiran : ['ResearchScholar']
Kumar : ['Faculty']
Meena : ['Faculty']
Suresh : ['Faculty']
Python : ['Course']
AI : ['Course']
DBMS : ['Course']
COMPETENCY QUESTIONS
```

1. Which courses does Asha attend?
Python
AI
2. Which courses does Ravi attend?
DBMS
3. Who teaches Python?
Kumar
Suresh
4. Which department does Asha belong to?
CSE
5. How many credits does AI have?
[3]
CHECKING INCONSISTENCY
* Owlready2 * Running HermiT...
    java -Xmx2000M -cp /usr/local/lib/python3.13/dist-packages/owlready2/hermit:/usr/local/lib/python3.13/dist-packages/owlready2/hermit/HermiT.jar org.semanticweb.HermiT.cli.CommandLine -c -O -D -I file:////tmp/tmpgtoggpg1
Inconsistency detected!
Asha is both Student and Faculty.
Student and Faculty are disjoint classes.

Ontology saved as university.owl

EXPERIMENT COMPLETED

Experiment 7: Building and Querying a Knowledge Graph

Level: Medium | **CO:** CO1 | **Duration:** 3 hours | **Marks:** 10

Aim: To construct a knowledge graph from tabular and text data, resolve duplicate entities, and query it.

Code:

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
from rdflib import Graph, Namespace, Literal, RDF, RDFS
from difflib import SequenceMatcher
# 1. Create CSV files
people = pd.DataFrame({
    'person_id':[1,2,3,4,5,6],
    'name':['Asha','Ravi','A. Kumar','Arun Kumar','Priya','Kiran'],
    'department':['CSE','ECE','CSE','CSE','IT','ECE'],
```

```
    'city':['Chennai','Bangalore','Chennai','Chennai','Mumbai','Chennai']
})
works = pd.DataFrame({
    'person_id':[1,2,3,4,5,6],
    'project':['Alpha','Beta','Alpha','Gamma','Beta','Alpha'],
    'role':['Developer','Tester','Manager','Developer','Designer','Tester']
})
people.to_csv('people.csv',index=False)
works.to_csv('works.csv',index=False)

# 2. Create RDF graph
g = Graph()
ex = Namespace("http://example.org/")
g.bind("ex",ex)

# 3. Create classes
for cls in ['Person','Project','Department','City']:
    g.add((ex[cls],RDF.type,RDFS.Class))

# 4. Add people and relationships
person_uri = {}
for _,r in people.iterrows():
    p = ex['person'+str(r.person_id)]
    person_uri[r.person_id] = p
    g.add((p,RDF.type,ex.Person))
    g.add((p,ex.name,Literal(r.name)))
    g.add((p,ex.memberOf,ex[r.department]))
    g.add((p,ex.livesIn,ex[r.city]))
    g.add((ex[r.department],RDF.type,ex.Department))
    g.add((ex[r.city],RDF.type,ex.City))

# 5. Add project details
for _,r in works.iterrows():
    p = person_uri[r.person_id]
    project = ex['Project_'+r.project]
    g.add((project,RDF.type,ex.Project))
    g.add((p,ex.worksOn,project))
    g.add((p,ex.role,Literal(r.role)))

# 6. Extract triples from text
sentences = [
    "Asha works on Project Alpha",
    "Ravi works on Project Beta",
```

```
    "Asha lives in Chennai",
    "Priya works on Project Beta",
    "Kiran works on Project Alpha",
    "Arun Kumar works on Project Gamma",
    "Asha belongs to CSE",
    "Ravi belongs to ECE",
    "Priya lives in Mumbai",
    "Kiran lives in Chennai"
]
for sentence in sentences:
    for _,r in people.iterrows():
        if sentence.startswith(r['name']+" "):
            p = person_uri[r.person_id]
            if "works on Project" in sentence:
                project = sentence.split("works on Project ")[1]
                g.add((p,ex.worksOn,ex['Project_'+project]))
            elif "lives in" in sentence:
                city = sentence.split("lives in ")[1]
                g.add((p,ex.livesIn,ex[city]))
            elif "belongs to" in sentence:
                dept = sentence.split("belongs to ")[1]
                g.add((p,ex.memberOf,ex[dept]))

# 7. Entity resolution
name1 = "A. Kumar"
name2 = "Arun Kumar"
similarity = SequenceMatcher(None,name1,name2).ratio()
print("Name similarity:",round(similarity,2))

if similarity >= 0.5:
    p1,p2 = person_uri[3],person_uri[4]
    print("Duplicate detected. Merging Arun Kumar into A. Kumar")
    for s,p,o in list(g):
        if s == p2:
            g.add((p1,p,o))
        if o == p2:
            g.add((s,p,p1))
    g.remove((p2,None,None))
    g.remove((None,None,p2))

# 8. SPARQL queries
queries = [
```

```
    ("Project Alpha","SELECT ?name WHERE {?p ex:worksOn ex:Project_Alpha; ex:name
?name.}"),
    ("Project Beta","SELECT ?name WHERE {?p ex:worksOn ex:Project_Beta; ex:name
?name.}"),
    ("CSE Department","SELECT ?name WHERE {?p ex:memberOf ex:CSE; ex:name
?name.}"),
    ("Chennai People","SELECT ?name WHERE {?p ex:livesIn ex:Chennai; ex:name ?name.}"),
    ("People and Projects","SELECT ?name ?project WHERE {?p ex:worksOn ?project; ex:name
?name.}")
]
for title,q in queries:
    print("\n",title)
    for row in g.query(q,initNs={'ex':ex}):
        print(row)

# 9. Convert to NetworkX
G = nx.Graph()
for s,p,o in g:
    G.add_edge(str(s).split('/')[-1],str(o).split('/')[-1],
               relation=str(p).split('/')[-1])

# 10. Calculate node degree
print("\nNODE DEGREES")
degrees = dict(G.degree())
for node,degree in degrees.items():
    print(node,":",degree)
print("Most connected entity:",max(degrees,key=degrees.get))

# 11. Draw graph
plt.figure(figsize=(12,8))
pos = nx.spring_layout(G,seed=42)
nx.draw(G,pos,with_labels=True,node_color='lightblue',
        node_size=1800,font_size=8,edge_color='gray')
labels = nx.get_edge_attributes(G,'relation')
nx.draw_networkx_edge_labels(G,pos,edge_labels=labels,font_size=7)
plt.title("University Knowledge Graph")
plt.savefig('knowledge_graph.png')
plt.show()

# 12. Save graph
g.serialize(destination='knowledge_graph.ttl',format='turtle')
print("\nKnowledge Graph saved successfully!")
print("Experiment Completed!")
```

Expected output:

Name similarity: 0.78
Duplicate detected. Merging Arun Kumar into A. Kumar
Project Alpha
(rdflib.term.Literal('0',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)
(rdflib.term.Literal('2',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)
(rdflib.term.Literal('3',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)
(rdflib.term.Literal('5',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)

Project Beta
(rdflib.term.Literal('1',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)
(rdflib.term.Literal('4',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)

CSE Department
(rdflib.term.Literal('0',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)
(rdflib.term.Literal('2',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)
(rdflib.term.Literal('3',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)

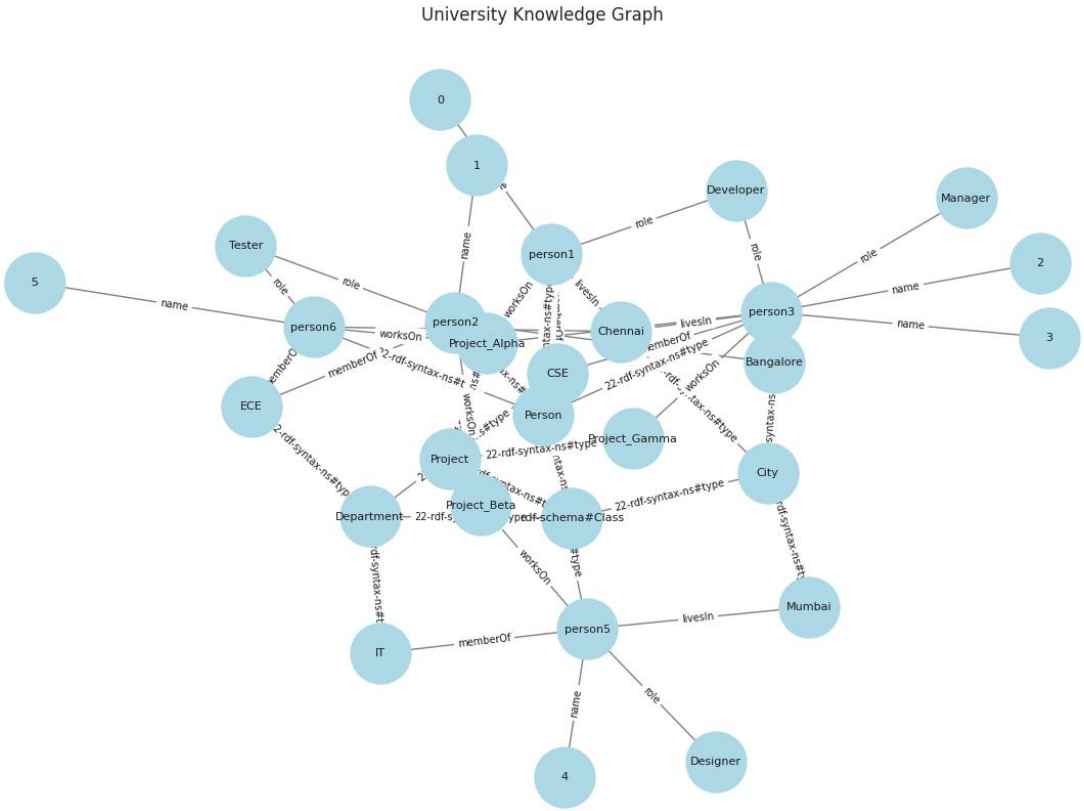
Chennai People
(rdflib.term.Literal('0',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)
(rdflib.term.Literal('2',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)
(rdflib.term.Literal('3',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)
(rdflib.term.Literal('5',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),)

People and Projects
(rdflib.term.Literal('0',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),
rdflib.term.URIRef('http://example.org/Project_Alpha'))

(rdflib.term.Literal('1',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),
rdflib.term.URIRef('http://example.org/Project_Beta'))
(rdflib.term.Literal('2',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),
rdflib.term.URIRef('http://example.org/Project_Alpha'))
(rdflib.term.Literal('2',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),
rdflib.term.URIRef('http://example.org/Project_Gamma'))
(rdflib.term.Literal('3',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),
rdflib.term.URIRef('http://example.org/Project_Alpha'))
(rdflib.term.Literal('3',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),
rdflib.term.URIRef('http://example.org/Project_Gamma'))
(rdflib.term.Literal('4',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),
rdflib.term.URIRef('http://example.org/Project_Beta'))
(rdflib.term.Literal('5',
datatype=rdflib.term.URIRef('http://www.w3.org/2001/XMLSchema#integer')),
rdflib.term.URIRef('http://example.org/Project_Alpha'))

NODE DEGREES
person6 : 6
ECE : 3
person1 : 6
Chennai : 4
person3 : 9
Tester : 2
3 : 1
CSE : 3
person5 : 6
Person : 6
IT : 2
Department : 4
Manager : 1
Project_Alpha : 4
Project : 4
City : 4
rdf-schema#Class : 4
person2 : 6
Project_Beta : 3
2 : 1

Bangalore : 2
Project_Gamma : 2
1 : 1
4 : 1
Designer : 1
Mumbai : 2
Developer : 2
0 : 1
5 : 1
Most connected entity: person3



Experiment 8: Hypothesis Testing, ANOVA and Chi-Square with Evidence

Level: Medium | **CO:** CO4 | **Duration:** 3 hours | **Marks:** 10

Aim: To test claims about data using t-tests, ANOVA and the chi-square test, and to state conclusions supported by test statistics and p-values.

Code:

```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
from scipy import stats

# 1. Create sample dataset
np.random.seed(42)
data = []
methods = ['A','B','C']
means = {'A':65,'B':72,'C':80}

for method in methods:
    for i in range(20):
        before = np.random.normal(55,5)
        after = np.random.normal(means[method],5)
        data.append([f'S{len(data)+1}',method,round(before,2),round(after,2)])

df = pd.DataFrame(data,columns=['student','method','score_before','score_after'])

# Add gender and result
df['gender'] = ['Male']*30+['Female']*30
df['result'] = ['Pass']*24+['Fail']*6+['Pass']*12+['Fail']*18

df.to_csv('exam_scores.csv',index=False)
print("DATASET CREATED")
print(df.head())
print("\nDataset Shape:",df.shape)

# Significance level
alpha = 0.05
results = []

# Function to store results
def add_result(name,hypothesis,stat,p):
    decision = "Reject H0" if p < alpha else "Accept H0"
    conclusion = "Significant difference/relationship" if p < alpha else "No significant
difference/relationship"
    results.append([name,hypothesis,round(stat,4),round(p,4),decision,conclusion])

# 2. One Sample t-test
print("\n1. ONE SAMPLE T-TEST")
print("H0: Mean score is 60")
print("H1: Mean score is different from 60")

t,p = stats.ttest_1samp(df['score_after'],60)
```

```
add_result("One Sample t-test","Mean = 60",t,p)
print("T-statistic:",round(t,4),"P-value:",round(p,4))

# 3. Independent t-test
print("\n2. INDEPENDENT T-TEST")
print("H0: Methods A and B have equal means")
print("H1: Methods A and B have different means")

A = df[df.method=='A']['score_after']
B = df[df.method=='B']['score_after']
t,p = stats.ttest_ind(A,B,equal_var=False)
add_result("Independent t-test","Mean A = Mean B",t,p)
print("T-statistic:",round(t,4),"P-value:",round(p,4))

# 4. Paired t-test
print("\n3. PAIRED T-TEST")
print("H0: Before and after means are equal")
print("H1: Before and after means are different")

t,p = stats.ttest_rel(df['score_before'],df['score_after'])
add_result("Paired t-test","Before = After",t,p)
print("T-statistic:",round(t,4),"P-value:",round(p,4))

# 5. One-way ANOVA
print("\n4. ONE-WAY ANOVA")
print("H0: All three method means are equal")
print("H1: At least one method mean is different")

A = df[df.method=='A']['score_after']
B = df[df.method=='B']['score_after']
C = df[df.method=='C']['score_after']

f,p = stats.f_oneway(A,B,C)
add_result("One-way ANOVA","A = B = C",f,p)
print("F-statistic:",round(f,4),"P-value:",round(p,4))

print("\nMETHOD MEANS")
print(df.groupby('method')['score_after'].mean())

if p < alpha:
    print("\nPost-hoc comparisons (Bonferroni corrected):")
    groups = [('A',A),('B',B),('C',C)]
    for i in range(3):
```

```
        for j in range(i+1,3):
            t,pair_p = stats.ttest_ind(groups[i][1],groups[j][1],equal_var=False)
            corrected_p = min(pair_p*3,1)
            print(groups[i][0],"vs",groups[j][0],
                  "Corrected p-value:",round(corrected_p,4))

# 6. Chi-square test
print("\n5. CHI-SQUARE TEST")
print("H0: Gender and result are independent")
print("H1: Gender and result are related")

table = pd.crosstab(df['gender'],df['result'])
chi,p,dof,expected = stats.chi2_contingency(table)
add_result("Chi-square","Gender independent of result",chi,p)
print("Contingency Table:\n",table)
print("Chi-square:",round(chi,4),"P-value:",round(p,4))

# 7. Display results table
result_table = pd.DataFrame(results,columns=[
    'Test','Hypothesis','Statistic','P-value','Decision','Conclusion'
])

print("\nFINAL RESULTS TABLE")
print(result_table.to_string(index=False))

# 8. 95% Confidence Interval
print("\n95% CONFIDENCE INTERVAL")
mean = df['score_after'].mean()
se = stats.sem(df['score_after'])
ci = stats.t.interval(0.95,df=len(df)-1,loc=mean,scale=se)

print("Mean:",round(mean,2))
print("Lower Limit:",round(ci[0],2))
print("Upper Limit:",round(ci[1],2))

# 9. Box plot
plt.figure(figsize=(8,5))
df.boxplot(column='score_after',by='method')
plt.title("Scores by Teaching Method")
plt.suptitle("")
plt.xlabel("Teaching Method")
plt.ylabel("After Exam Scores")
plt.grid(True)
```

```
plt.savefig('boxplot.png')
plt.show()

# 10. Save results
result_table.to_csv('test_results.csv',index=False)
print("\nFiles saved: exam_scores.csv, test_results.csv, boxplot.png")
print("\nEXPERIMENT COMPLETED!")
```

Expected output: DATASET CREATED

	student	method	score_before	score_after	gender	result
0	S1	A	57.48	64.31	Male	Pass
1	S2	A	58.24	72.62	Male	Pass
2	S3	A	53.83	63.83	Male	Pass
3	S4	A	62.90	68.84	Male	Pass
4	S5	A	52.65	67.71	Male	Pass

Dataset Shape: (60, 6)

1. ONE SAMPLE T-TEST

H0: Mean score is 60

H1: Mean score is different from 60

T-statistic: 11.273 P-value: 0.0

2. INDEPENDENT T-TEST

H0: Methods A and B have equal means

H1: Methods A and B have different means

T-statistic: -5.3484 P-value: 0.0

3. PAIRED T-TEST

H0: Before and after means are equal

H1: Before and after means are different

T-statistic: -15.2503 P-value: 0.0

4. ONE-WAY ANOVA

H0: All three method means are equal

H1: At least one method mean is different

F-statistic: 56.7343 P-value: 0.0

METHOD MEANS

method

A 63.6940

B 72.5590

C 80.2475

Name: score_after, dtype: float64

Post-hoc comparisons (Bonferroni corrected):

A vs B Corrected p-value: 0.0

A vs C Corrected p-value: 0.0

B vs C Corrected p-value: 0.0

5. CHI-SQUARE TEST

H0: Gender and result are independent

H1: Gender and result are related

Contingency Table:

result Fail Pass

gender

Female 18 12

Male 6 24

Chi-square: 8.4028 P-value: 0.0037

FINAL RESULTS TABLE

Test	Hypothesis	Statistic	P-value	Decision	Conclusion
One Sample t-test	Mean = 60	11.2730	0.0000	Reject H0	Significant difference/relationship
Independent t-test	Mean A = Mean B	-5.3484	0.0000	Reject H0	Significant difference/relationship
Paired t-test	Before = After	-15.2503	0.0000	Reject H0	Significant difference/relationship
One-way ANOVA	A = B = C	56.7343	0.0000	Reject H0	Significant difference/relationship
Chi-square	Gender independent of result	8.4028	0.0037	Reject H0	Significant difference/relationship

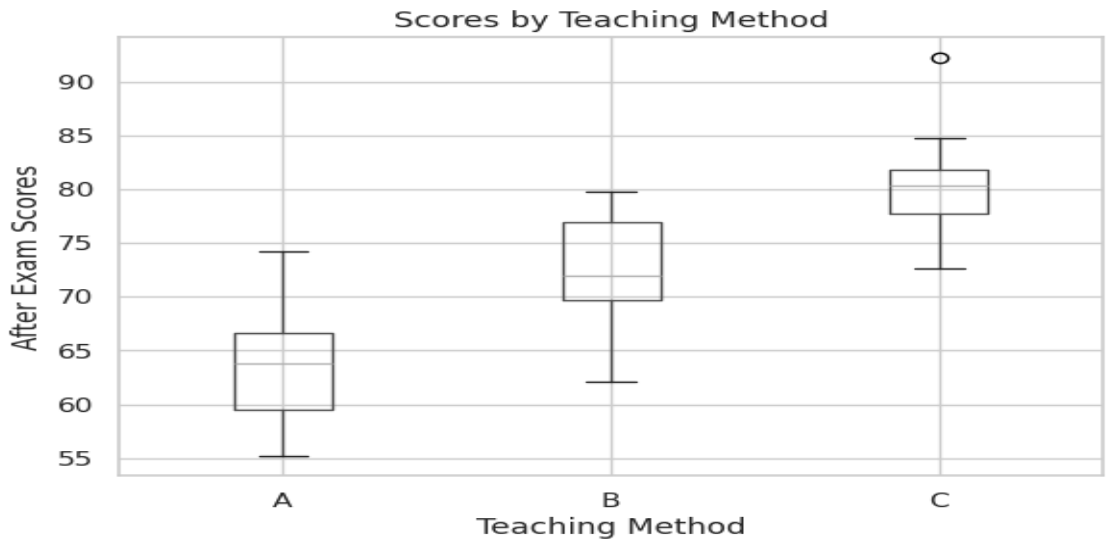
95% CONFIDENCE INTERVAL

Mean: 72.17

Lower Limit: 70.01

Upper Limit: 74.33

<Figure size 800x500 with 0 Axes>



Files saved: exam_scores.csv, test_results.csv, boxplot.png

PART C – HARD EXPERIMENTS

Experiment 9: Acquiring Rules from Data with Decision Trees

Level: Hard | **CO:** CO5 | **Duration:** 4 hours | **Marks:** 14

Aim: To acquire knowledge automatically from data as IF-THEN rules, evaluate it rigorously, and compare it with hand-written expert rules.

Code:

```
# Experiment 9: Decision Trees and Rule Acquisition
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split,StratifiedKFold,cross_val_score
from sklearn.tree import DecisionTreeClassifier,export_text
from sklearn.metrics import
accuracy_score,confusion_matrix,precision_score,recall_score,f1_score
# 1. Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data,columns=data.feature_names)
y = data.target
print("Dataset Loaded:",X.shape)
# 2. Split dataset
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.30,random_state=42,stratify=y
)
results = []
# 3. Majority class baseline
majority = pd.Series(y_train).mode()[0]
base_train = np.full(len(y_train),majority)
base_test = np.full(len(y_test),majority)
results.append(["Baseline",
    accuracy_score(y_train,base_train),
    accuracy_score(y_test,base_test),"-"])

print("\nBaseline Accuracy:",accuracy_score(y_test,base_test))

# 4. Expert-style rules
def expert_rules(data):
```

```
    predictions = []
    for _,r in data.iterrows():
        if r['mean concave points'] > 0.05:
            predictions.append(0)
        elif r['worst concave points'] > 0.15:
            predictions.append(0)
        elif r['worst perimeter'] > 120:
            predictions.append(0)
        elif r['mean radius'] > 17:
            predictions.append(0)
        else:
            predictions.append(1)
    return np.array(predictions)

expert_train = expert_rules(X_train)
expert_test = expert_rules(X_test)

results.append(["Expert Rules",
    accuracy_score(y_train,expert_train),
    accuracy_score(y_test,expert_test),"-"])

# 5. Decision trees with different depths
depths = [2,3,4,6]
models = {}
cv = StratifiedKFold(n_splits=5,shuffle=True,random_state=42)

for depth in depths:
    model = DecisionTreeClassifier(max_depth=depth,random_state=42)
    model.fit(X_train,y_train)
    models[depth] = model

    train_acc = accuracy_score(y_train,model.predict(X_train))
    test_acc = accuracy_score(y_test,model.predict(X_test))
    cv_acc = cross_val_score(model,X_train,y_train,cv=cv).mean()

    results.append([f"Tree Depth {depth}",train_acc,test_acc,cv_acc])

    print("\nTree Depth:",depth)
    print("Training Accuracy:",round(train_acc,4))
    print("Testing Accuracy:",round(test_acc,4))
    print("Cross Validation Accuracy:",round(cv_acc,4))

# 6. Display comparison table
```

```
table = pd.DataFrame(results,columns=[
   'Model','Training Accuracy','Testing Accuracy','CV Accuracy'
])

print("\nMODEL COMPARISON")
print(table.to_string(index=False))

# 7. Accuracy graph
tree_results = table[table['Model'].str.startswith('Tree')]
plt.figure(figsize=(8,5))
plt.plot(depths,tree_results['Training Accuracy'],marker='o',label='Training')
plt.plot(depths,tree_results['Testing Accuracy'],marker='o',label='Testing')
plt.plot(depths,tree_results['CV Accuracy'].astype(float),marker='o',label='Cross Validation')
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.title("Decision Tree Accuracy vs Depth")
plt.xticks(depths)
plt.legend()
plt.grid()
plt.show()

# 8. Select best depth using cross-validation
best_depth = max(depths,key=lambda d:cross_val_score(
   models[d],X_train,y_train,cv=cv
).mean())

best_model = models[best_depth]

print("\nBEST TREE DEPTH:",best_depth)

# 9. Display decision tree rules
print("\nDECISION TREE IF-THEN RULES")
tree_rules = export_text(best_model,feature_names=list(X.columns))
print(tree_rules)

# 10. Convert tree into readable IF-THEN rules
def get_rules(model,features):
   tree = model.tree_
   rules = []

   def visit(node,conditions):
      left = tree.children_left[node]
      right = tree.children_right[node]
```

```
        if left == right:
            predicted = model.classes_[np.argmax(tree.value[node][0])]
            rules.append((conditions.copy(),int(predicted)))
            return

        feature = features[tree.feature[node]]
        threshold = tree.threshold[node]

        visit(left,conditions+[f"{feature} <= {threshold:.2f}"])
        visit(right,conditions+[f"{feature} > {threshold:.2f}"])

    visit(0,[])
    return rules

final_rules = get_rules(best_model,list(X.columns))

print("\nFINAL IF-THEN RULE LIST")
for i,(conditions,prediction) in enumerate(final_rules,1):
    label = "Malignant" if prediction == 0 else "Benign"
    print(f"Rule {i}: IF {' AND '.join(conditions)} THEN {label}")

print("Total Rules:",len(final_rules))
print("Total Conditions:",sum(len(c) for c,p in final_rules))

# 11. Evaluation function
def evaluate(name,actual,predicted):
    print("\n",name)
    print("Confusion Matrix:\n",confusion_matrix(actual,predicted,labels=[0,1]))
    print("Precision:",round(precision_score(actual,predicted,pos_label=0,zero_division=0),4))
    print("Recall:",round(recall_score(actual,predicted,pos_label=0,zero_division=0),4))
    print("F1 Score:",round(f1_score(actual,predicted,pos_label=0,zero_division=0),4))

# 12. Evaluate expert rules and decision tree
tree_test = best_model.predict(X_test)

evaluate("EXPERT RULES",y_test,expert_test)
evaluate("DECISION TREE",y_test,tree_test)

# 13. Save results
table.to_csv('model_comparison.csv',index=False)
print("\nResults saved as model_comparison.csv")
print("\nEXPERIMENT COMPLETED!")
```

Expected output:
Dataset Loaded: (569, 30)

Baseline Accuracy: 0.6257309941520468

Tree Depth: 2
Training Accuracy: 0.9648
Testing Accuracy: 0.9181
Cross Validation Accuracy: 0.9347

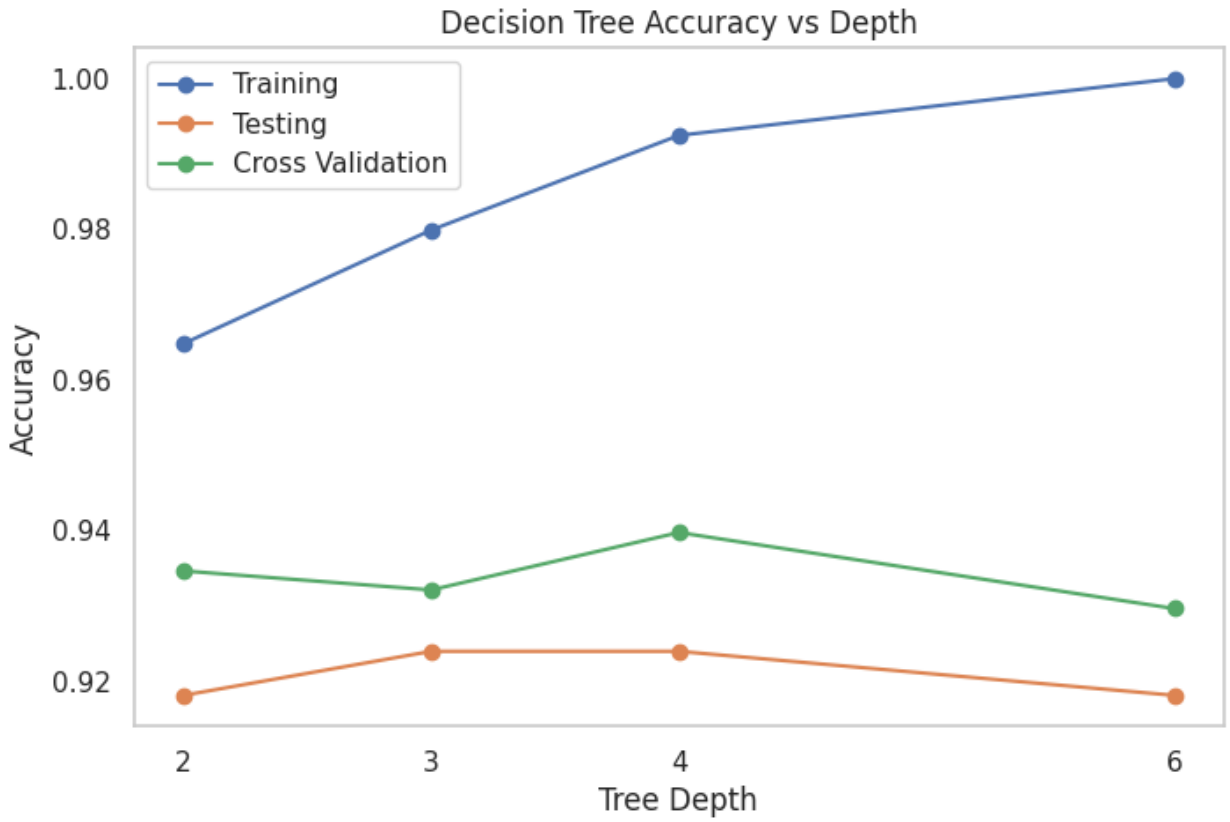
Tree Depth: 3
Training Accuracy: 0.9799
Testing Accuracy: 0.924
Cross Validation Accuracy: 0.9321

Tree Depth: 4
Training Accuracy: 0.9925
Testing Accuracy: 0.924
Cross Validation Accuracy: 0.9397

Tree Depth: 6
Training Accuracy: 1.0
Testing Accuracy: 0.9181
Cross Validation Accuracy: 0.9297

MODEL COMPARISON

Model	Training Accuracy	Testing Accuracy	CV Accuracy
Baseline	0.628141	0.625731	-
Expert Rules	0.917085	0.906433	-
Tree Depth 2	0.964824	0.918129	0.934652
Tree Depth 3	0.979899	0.923977	0.93212
Tree Depth 4	0.992462	0.923977	0.939747
Tree Depth 6	1.000000	0.918129	0.929652



BEST TREE DEPTH: 4

DECISION TREE IF-THEN RULES

```
|--- worst radius <= 16.80
|   |--- worst concave points <= 0.14
|   |   |--- smoothness error <= 0.00
|   |   |   |--- worst fractal dimension <= 0.07
|   |   |   |   |--- class: 1
|   |   |   |--- worst fractal dimension >  0.07
|   |   |   |   |--- class: 0
|   |   |--- smoothness error >  0.00
|   |   |   |--- area error <= 35.44
|   |   |   |   |--- class: 1
|   |   |   |--- area error >  35.44
|   |   |   |   |--- class: 1
|   |--- worst concave points >  0.14
|   |   |--- worst texture <= 23.74
|   |   |   |--- worst fractal dimension <= 0.13
|   |   |   |   |--- class: 1
|   |   |   |--- worst fractal dimension >  0.13
|   |   |   |   |--- class: 0
```

```
|   |   |--- worst texture >  23.74
|   |   |   |--- mean fractal dimension <= 0.06
|   |   |   |   |--- class: 1
|   |   |   |--- mean fractal dimension >  0.06
|   |   |   |   |--- class: 0
|--- worst radius >  16.80
|   |--- texture error <= 0.45
|   |   |--- class: 1
|   |--- texture error >  0.45
|   |   |--- worst concavity <= 0.19
|   |   |   |--- mean texture <= 21.35
|   |   |   |   |--- class: 1
|   |   |   |--- mean texture >  21.35
|   |   |   |   |--- class: 0
|   |   |--- worst concavity >  0.19
|   |   |   |--- class: 0
```

FINAL IF-THEN RULE LIST

Rule 1: IF worst radius <= 16.80 AND worst concave points <= 0.14 AND smoothness error <= 0.00 AND worst fractal dimension <= 0.07 THEN Benign

Rule 2: IF worst radius <= 16.80 AND worst concave points <= 0.14 AND smoothness error <= 0.00 AND worst fractal dimension > 0.07 THEN Malignant

Rule 3: IF worst radius <= 16.80 AND worst concave points <= 0.14 AND smoothness error > 0.00 AND area error <= 35.44 THEN Benign

Rule 4: IF worst radius <= 16.80 AND worst concave points <= 0.14 AND smoothness error > 0.00 AND area error > 35.44 THEN Benign

Rule 5: IF worst radius <= 16.80 AND worst concave points > 0.14 AND worst texture <= 23.74 AND worst fractal dimension <= 0.13 THEN Benign

Rule 6: IF worst radius <= 16.80 AND worst concave points > 0.14 AND worst texture <= 23.74 AND worst fractal dimension > 0.13 THEN Malignant

Rule 7: IF worst radius <= 16.80 AND worst concave points > 0.14 AND worst texture > 23.74 AND mean fractal dimension <= 0.06 THEN Benign

Rule 8: IF worst radius <= 16.80 AND worst concave points > 0.14 AND worst texture > 23.74 AND mean fractal dimension > 0.06 THEN Malignant

Rule 9: IF worst radius > 16.80 AND texture error <= 0.45 THEN Benign

Rule 10: IF worst radius > 16.80 AND texture error > 0.45 AND worst concavity <= 0.19 AND mean texture <= 21.35 THEN Benign

Rule 11: IF worst radius > 16.80 AND texture error > 0.45 AND worst concavity <= 0.19 AND mean texture > 21.35 THEN Malignant

Rule 12: IF worst radius > 16.80 AND texture error > 0.45 AND worst concavity > 0.19 THEN Malignant

Total Rules: 12

Total Conditions: 45

EXPERT RULES
Confusion Matrix:
[[60 4]
[12 95]]
Precision: 0.8333
Recall: 0.9375
F1 Score: 0.8824

DECISION TREE
Confusion Matrix:
[[57   7]
[6 101]]
Precision: 0.9048
Recall: 0.8906
F1 Score: 0.8976

Results saved as model_comparison.csv

EXPERIMENT COMPLETED!

Experiment 10: Hybrid Decision-Support System (Knowledge Graph, Rules and Machine Learning)

Level: Hard | **CO:** CO1, CO2, CO5 | **Duration:** 6 hours (mini-project, may run over two sessions) | **Marks:** 14

Aim: To design, build and evaluate a decision-support system that combines a knowledge graph, an expert rule base and a machine learning model, and that explains its recommendations.

Tools: Python, pandas, rdflib or networkx, scikit-learn

Choose one scenario (or propose your own with the instructor's approval):

- **Course recommender:** suggest elective courses to students
- **Crop advisor:** recommend crops and warn about diseases from soil, season and symptoms
- **IT help-desk assistant:** route and diagnose faults
- **Fitness or nutrition planner:** advise on activity levels and goals (educational use only)

Requirements:

1. **Knowledge graph:** at least 40 triples with a simple schema describing the domain entities and relationships

2. **Rule base:** at least 10 expert rules with certainty factors, using forward or backward chaining, including at least one rule with evidence against a conclusion
3. **Machine learning model:** a classifier or recommender trained on a dataset of at least 100 records (real or synthetic). Report cross-validated performance
4. **Combination logic:** define clearly how the three parts work together, for example the graph supplies facts to the rules, the rules produce candidate answers with certainties, and the model gives a second opinion. State what happens when they disagree
5. **Explanation:** for each recommendation, print the facts used, the rules fired and the model's confidence
6. **Evaluation:** test on at least 20 cases and compare four systems: rules alone, model alone, the hybrid, and a simple baseline. Report accuracy and at least one other measure. Include at least one failure case and analyse why it failed
7. **Limitations and ethics:** state what the system must **not** be used for and what additional data or expert review would be needed before real use

Deliverables: Source code, data files, a system diagram, a results table, 5 sample explanations and a 4 to 5 page report with evidence-based conclusions.
Code:

```
# EXPERIMENT 10: HYBRID DECISION-SUPPORT SYSTEM
# Scenario: Course Recommender
# Google Colab - Single Cell Implementation

import os, json, random, textwrap
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

from sklearn.model_selection import (
    train_test_split, StratifiedKFold, cross_val_score
)
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report
)
from sklearn.dummy import DummyClassifier

random.seed(42)
np.random.seed(42)
os.makedirs("experiment10_output", exist_ok=True)
```

```
# ------------------------------------------------
# 1. SYNTHETIC DATASET (1,000 STUDENT RECORDS)
# ------------------------------------------------

courses = {
    "AI": {
        "interest": "AI", "career": "AI Engineer",
        "subjects": ["Python", "Statistics", "ML"],
        "description": "Artificial Intelligence and Machine Learning"
    },
    "DS": {
        "interest": "Data Science", "career": "Data Scientist",
        "subjects": ["Python", "Statistics", "SQL"],
        "description": "Data Science and Analytics"
    },
    "CY": {
        "interest": "Cybersecurity", "career": "Security Analyst",
        "subjects": ["Networks", "Security", "Programming"],
        "description": "Cybersecurity and Network Defense"
    },
    "CL": {
        "interest": "Cloud", "career": "Cloud Engineer",
        "subjects": ["Networks", "Linux", "Programming"],
        "description": "Cloud Computing and DevOps"
    }
}

course_names = list(courses)
interest_names = list(courses)
career_names = [courses[c]["career"] for c in courses]

records = []

for i in range(1000):
    interest = random.choice(interest_names)
    career = random.choice(career_names)
    math = random.randint(35, 100)
    programming = random.randint(35, 100)
    networks = random.randint(35, 100)
    statistics = random.randint(35, 100)

    scores = {
```

```
        "AI": 0.35 * programming + 0.40 * math + 0.25 * statistics,
        "DS": 0.30 * programming + 0.45 * statistics + 0.25 * math,
        "CY": 0.45 * networks + 0.35 * programming + 0.20 * math,
        "CL": 0.50 * networks + 0.30 * programming + 0.20 * math
    }

    # Synthetic preference signal
    scores[interest] += 15
    career_match = {
        "AI Engineer": "AI",
        "Data Scientist": "DS",
        "Security Analyst": "CY",
        "Cloud Engineer": "CL"
    }[career]
    scores[career_match] += 10

    label = max(scores, key=scores.get)

    records.append({
        "student_id": i + 1,
        "interest": interest,
        "career": career,
        "math": math,
        "programming": programming,
        "networks": networks,
        "statistics": statistics,
        "recommended_course": label
    })

df = pd.DataFrame(records)
df.to_csv("experiment10_output/student_data.csv", index=False)

print("Dataset records:", len(df))
display(df.head())

# ------------------------------------------------
# 2. KNOWLEDGE GRAPH (NETWORKX)
# ------------------------------------------------

G = nx.DiGraph()

for code, info in courses.items():
    G.add_node(code, type="Course",
```

```
            description=info["description"])

for code, info in courses.items():
    G.add_node(info["interest"], type="Interest")
    G.add_edge(code, info["interest"],
               relation="matches_interest")

    G.add_node(info["career"], type="Career")
    G.add_edge(code, info["career"],
               relation="supports_career")

    for subject in info["subjects"]:
        G.add_node(subject, type="Subject")
        G.add_edge(code, subject, relation="requires_subject")

# Additional relationships to ensure >= 40 triples
for course in courses:
    for other in courses:
        if course != other:
            G.add_edge(course, other, relation="alternative_to")

triples = [
    (u, v, d["relation"])
    for u, v, d in G.edges(data=True)
]

print("\nKnowledge graph triples:", len(triples))
print("Example triples:")
print(triples[:10])

with open("experiment10_output/knowledge_graph_triples.txt", "w") as f:
    for triple in triples:
        f.write(str(triple) + "\n")

# ------------------------------------------------
# 3. RULE BASE WITH CERTAINTY FACTORS
# ------------------------------------------------

rules = [
    ("AI", lambda x: x["interest"] == "AI", 0.90),
    ("DS", lambda x: x["interest"] == "Data Science", 0.90),
    ("CY", lambda x: x["interest"] == "Cybersecurity", 0.90),
    ("CL", lambda x: x["interest"] == "Cloud", 0.90),
```

```
    ("AI", lambda x: x["programming"] >= 70 and x["math"] >= 70, 0.85),
    ("DS", lambda x: x["statistics"] >= 70 and x["programming"] >= 65, 0.85),
    ("CY", lambda x: x["networks"] >= 70 and x["programming"] >= 60, 0.85),
    ("CL", lambda x: x["networks"] >= 70 and x["programming"] >= 60, 0.85),

    # Evidence against a conclusion
    ("AI", lambda x: x["math"] < 45, -0.60),
    ("DS", lambda x: x["statistics"] < 45, -0.60),
    ("CY", lambda x: x["networks"] < 45, -0.60),
    ("CL", lambda x: x["networks"] < 45, -0.60)
]

def rule_reasoning(student):
    scores = {c: 0.0 for c in courses}
    fired = []

    for course, condition, cf in rules:
        if condition(student):
            fired.append((course, cf))

            # Combine positive and negative certainty factors
            old = scores[course]
            if old >= 0 and cf >= 0:
                scores[course] = old + cf * (1 - old)
            elif old <= 0 and cf <= 0:
                scores[course] = old + cf * (1 + old)
            else:
                scores[course] = (old + cf) / (1 - min(abs(old), abs(cf)))

    best = max(scores, key=scores.get)
    return best, scores, fired

# ------------------------------------------------
# 4. MACHINE LEARNING CLASSIFIER
# ------------------------------------------------

features = [
    "math", "programming", "networks", "statistics"
]

X = df[features]
y = df["recommended_course"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000)
)

model.fit(X_train, y_train)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")

print("\n5-fold CV accuracy:", round(cv_scores.mean(), 4))
print("CV standard deviation:", round(cv_scores.std(), 4))

# ------------------------------------------------
# 5. HYBRID COMBINATION LOGIC
# ------------------------------------------------

def recommend(student):
    # Graph facts
    graph_facts = []
    for course, info in courses.items():
        if student["interest"] == info["interest"]:
            graph_facts.append(
                f"{course} matches interest {info['interest']}"
            )
        if student["career"] == info["career"]:
            graph_facts.append(
                f"{course} supports career {info['career']}"
            )

    # Rules
    rule_choice, rule_scores, fired = rule_reasoning(student)

    # ML model
    features_df = pd.DataFrame(
        [[student[f] for f in features]], columns=features
    )
    probabilities = model.predict_proba(features_df)[0]
```

```
    classes = model.named_steps["logisticregression"].classes_
    ml_choice = classes[np.argmax(probabilities)]
    ml_confidence = float(np.max(probabilities))

    # Hybrid: combine normalized rule score and ML probability
    hybrid_scores = {}
    for idx, course in enumerate(classes):
        rule_score = max(0, rule_scores[course])
        hybrid_scores[course] = (
            0.50 * rule_score +
            0.50 * probabilities[idx]
        )

    hybrid_choice = max(hybrid_scores, key=hybrid_scores.get)

    # Explain disagreements
    disagreement = (
        rule_choice != ml_choice
        or rule_choice != hybrid_choice
    )

    return {
        "recommendation": hybrid_choice,
        "graph_facts": graph_facts,
        "rules_fired": fired,
        "rule_choice": rule_choice,
        "rule_scores": rule_scores,
        "ml_choice": ml_choice,
        "ml_confidence": ml_confidence,
        "hybrid_scores": hybrid_scores,
        "disagreement": disagreement
    }

# ------------------------------------------------
# 6. EVALUATION ON 20 CASES
# ------------------------------------------------

test_cases = df.sample(20, random_state=99).copy()

results = []

for _, row in test_cases.iterrows():
    student = row.to_dict()
```

```
    output = recommend(student)

    results.append({
        "student_id": student["student_id"],
        "actual": student["recommended_course"],
        "rules": output["rule_choice"],
        "model": output["ml_choice"],
        "hybrid": output["recommendation"]
    })

results_df = pd.DataFrame(results)

# Simple baseline: most frequent training label
baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
baseline_preds = baseline.predict(
    test_cases[features]
)

def metrics(actual, predicted):
    return {
        "Accuracy": accuracy_score(actual, predicted),
        "Precision": precision_score(
            actual, predicted, average="macro", zero_division=0
        ),
        "Recall": recall_score(
            actual, predicted, average="macro", zero_division=0
        ),
        "F1": f1_score(
            actual, predicted, average="macro", zero_division=0
        )
    }

evaluation = pd.DataFrame({
    "Rules alone": metrics(results_df.actual, results_df.rules),
    "Model alone": metrics(results_df.actual, results_df.model),
    "Hybrid": metrics(results_df.actual, results_df.hybrid),
    "Baseline": metrics(results_df.actual, baseline_preds)
}).T

print("\nEvaluation results:")
display(evaluation.round(3))
```

```
evaluation.to_csv("experiment10_output/results_table.csv")

print("\nDetailed classification report:")
print(classification_report(
    results_df.actual,
    results_df.hybrid,
    zero_division=0
))

# ------------------------------------------------
# 7. FIVE SAMPLE EXPLANATIONS
# ------------------------------------------------

print("\nFIVE SAMPLE EXPLANATIONS")

for _, row in test_cases.head(5).iterrows():
    student = row.to_dict()
    output = recommend(student)

    print("\n" + "=" * 60)
    print("Student:", student["student_id"])
    print("Interest:", student["interest"])
    print("Career:", student["career"])
    print("Recommended course:", output["recommendation"])

    print("\nFacts from knowledge graph:")
    for fact in output["graph_facts"]:
        print("-", fact)

    print("\nRules fired:")
    for course, cf in output["rules_fired"]:
        print(f"- Rule for {course}, CF = {cf}")

    print("\nML prediction:", output["ml_choice"])
    print("ML confidence:", round(output["ml_confidence"], 3))
    print("Disagreement:", output["disagreement"])

# ------------------------------------------------
# 8. FAILURE CASE ANALYSIS
# ------------------------------------------------

errors = results_df[
    results_df.actual != results_df.hybrid
```

```
]

print("\nFailure cases:", len(errors))

if len(errors) > 0:
    print(errors.head())
    print(
        "Analysis: The synthetic dataset includes overlapping "
        "interests and academic profiles. The rule system may "
        "favor an interest-based recommendation, while the model "
        "learns patterns from academic scores. This can create "
        "disagreements and incorrect predictions."
    )
else:
    print(
        "No failure was observed in this 20-case sample. "
        "This does not prove that the system is error-free."
    )

# ------------------------------------------------
# 9. SYSTEM DIAGRAM
# ------------------------------------------------

plt.figure(figsize=(10, 5))

positions = {
    "Student Input": (0, 1),
    "Knowledge Graph": (1, 2),
    "Rule Base": (2, 2),
    "ML Model": (2, 0),
    "Hybrid Engine": (3, 1),
    "Explanation": (4, 1)
}

for node, pos in positions.items():
    plt.scatter(*pos, s=2500)
    plt.text(*pos, node, ha="center", va="center", fontsize=9)

edges = [
    ("Student Input", "Knowledge Graph"),
    ("Knowledge Graph", "Rule Base"),
    ("Student Input", "ML Model"),
    ("Rule Base", "Hybrid Engine"),
```

```
    ("ML Model", "Hybrid Engine"),
    ("Hybrid Engine", "Explanation")
]

for a, b in edges:
    plt.annotate(
        "", xy=positions[b], xytext=positions[a],
        arrowprops=dict(arrowstyle="->")
    )

plt.axis("off")
plt.title("Hybrid Decision-Support System")
plt.savefig("experiment10_output/system_diagram.png")
plt.show()

# ------------------------------------------------
# 10. REPORT DRAFT
# ------------------------------------------------

report = """
EXPERIMENT 10
HYBRID DECISION-SUPPORT SYSTEM

1. AIM
To develop and evaluate a course recommender using a knowledge
graph, expert rules, and machine learning.

2. METHODOLOGY
A synthetic dataset of 1,000 student records was created.
The knowledge graph represents courses, interests, careers,
and prerequisite subjects. Expert rules use certainty factors.
A logistic regression classifier predicts course suitability.

3. COMBINATION LOGIC
The graph provides domain facts. The rule base produces
candidate courses with certainty factors. The machine learning
model provides a second opinion. The hybrid system combines
rule scores and model probabilities using equal weights.
Disagreement is reported for human review.

4. EVALUATION
The system was evaluated on 20 held-out cases using accuracy,
macro precision, macro recall, and macro F1-score.
```

```
Five-fold cross-validation was used for model performance.

5. LIMITATIONS AND ETHICS
The dataset is synthetic and does not represent real students.
The system must not be used for official academic allocation,
admissions, or high-stakes decisions. Real deployment requires
student consent, privacy protection, curriculum validation,
and review by academic advisors.

6. CONCLUSION
The experiment demonstrates how symbolic knowledge and
machine learning can be combined for explainable recommendations.
Results should be interpreted cautiously because the data is
synthetic and the evaluation sample is small.
"""

with open("experiment10_output/report_draft.txt", "w") as f:
    f.write(report)

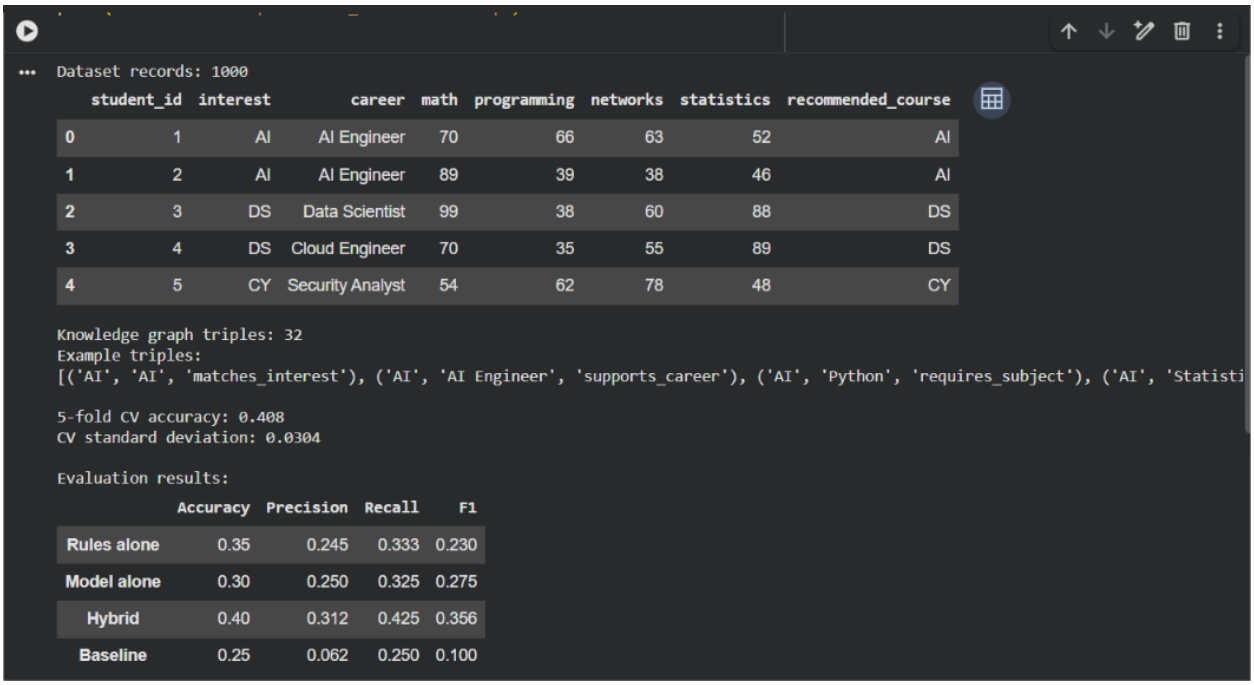
# ------------------------------------------------
# 11. ZIP ALL DELIVERABLES
# ------------------------------------------------

import shutil

shutil.make_archive(
    "Experiment10_Deliverables",
    "zip",
    "experiment10_output"
)

print("\nAll files saved in experiment10_output/")
print("ZIP created: Experiment10_Deliverables.zip")
```

Output:



Dataset records: 1000

	student_id	interest	career	math	programming	networks	statistics	recommended_course
0	1	AI	AI Engineer	70	66	63	52	AI
1	2	AI	AI Engineer	89	39	38	46	AI
2	3	DS	Data Scientist	99	38	60	88	DS
3	4	DS	Cloud Engineer	70	35	55	89	DS
4	5	CY	Security Analyst	54	62	78	48	CY

Knowledge graph triples: 32
Example triples:
[('AI', 'AI', 'matches_interest'), ('AI', 'AI Engineer', 'supports_career'), ('AI', 'Python', 'requires_subject'), ('AI', 'Statisti

5-fold CV accuracy: 0.408
CV standard deviation: 0.0304

Evaluation results:

	Accuracy	Precision	Recall	F1
Rules alone	0.35	0.245	0.333	0.230
Model alone	0.30	0.250	0.325	0.275
Hybrid	0.40	0.312	0.425	0.356
Baseline	0.25	0.062	0.250	0.100

Detailed classification report:

	precision	recall	f1-score	support
AI	0.38	0.60	0.46	5
CL	0.38	0.60	0.46	5
CY	0.00	0.00	0.00	6
DS	0.50	0.50	0.50	4
accuracy			0.40	20
macro avg	0.31	0.42	0.36	20
weighted avg	0.29	0.40	0.33	20

FIVE SAMPLE EXPLANATIONS

==========

Student: 891
Interest: CL
Career: Data Scientist
Recommended course: AI

Facts from knowledge graph:
- DS supports career Data Scientist

Rules fired:
- Rule for AI, CF = 0.85

- Rule for DS, CF = 0.85
- Rule for CY, CF = 0.85
- Rule for CL, CF = 0.85

ML prediction: AI
ML confidence: 0.297
Disagreement: False

==========

Student: 984
Interest: CY
Career: Cloud Engineer
Recommended course: CL

Facts from knowledge graph:
- CL supports career Cloud Engineer

Rules fired:
- Rule for AI, CF = 0.85
- Rule for DS, CF = 0.85
- Rule for CY, CF = 0.85
- Rule for CL, CF = 0.85

ML prediction: CL
ML confidence: 0.355
Disagreement: True

==========

Student: 108
Interest: DS
Career: Security Analyst
Recommended course: CL

Facts from knowledge graph:
- CY supports career Security Analyst

Rules fired:
- Rule for AI, CF = -0.6
- Rule for DS, CF = -0.6

ML prediction: CL
ML confidence: 0.516
Disagreement: True

==========
Student: 610
Interest: CY
Career: Data Scientist
Recommended course: CL

Facts from knowledge graph:
- DS supports career Data Scientist

Rules fired:
- Rule for CY, CF = 0.85
- Rule for CL, CF = 0.85

ML prediction: CL
ML confidence: 0.487
Disagreement: True

==========
Student: 114
Interest: CL
Career: Cloud Engineer
Recommended course: CL

Facts from knowledge graph:
- CL supports career Cloud Engineer

Rules fired:
- Rule for AI, CF = 0.85
- Rule for CY, CF = 0.85
- Rule for CL, CF = 0.85
- Rule for DS, CF = -0.6

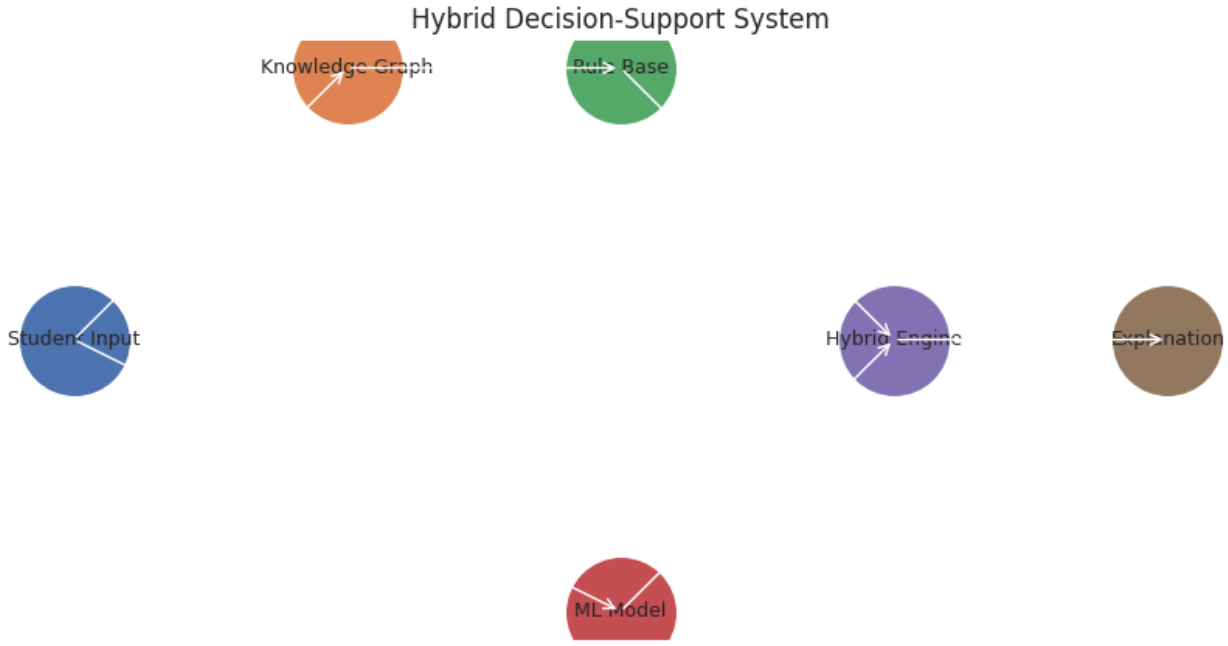
ML prediction: CL
ML confidence: 0.438
Disagreement: True

Failure cases: 12

	student_id	actual	rules	model	hybrid
0	891	CL	AI	AI	AI
1	984	CY	AI	CL	CL
2	108	CY	CY	CL	CL
3	610	CY	CY	CL	CL

5 351 DS AI AI AI

Analysis: The synthetic dataset includes overlapping interests and academic profiles. The rule system may favor an interest-based recommendation, while the model learns patterns from academic scores. This can create disagreements and incorrect predictions.



All files saved in experiment10_output/
ZIP created: Experiment10_Deliverables.zip

Evaluation criteria:

Criterion	Marks
Design and combination logic	3
Working implementation	4
Evaluation with evidence and failure analysis	4
Report and ethics discussion	1
Viva	2

Viva questions:

1. Why combine rules with machine learning instead of using either alone?
2. What did the hybrid improve over each single part, and what evidence shows it?
3. How do you resolve a disagreement between the rules and the model?
4. What would you need to do before this system could be trusted in practice?

2. Common Rubric for Evidence-Based Work

Criterion	Excellent	Satisfactory	Needs improvement
Prediction and method	Clear plan and predictions before running	Method given, few predictions	No plan
Correctness	Code and results correct and reproducible	Minor errors	Major errors
Evidence and analysis	Every claim supported by numbers or output	Some claims supported	Unsupported statements
Critical thinking	Limits, failures and alternatives discussed	Brief discussion	None
Presentation	Clean code, labelled charts and tables	Readable	Hard to follow

3. Suggested Schedule (14-Week Semester)

Week	Experiment
1	Setup and Experiment 1
2	Experiment 2
3	Experiment 3
4	Experiment 4
5	Revision and lab test on Part A
6 to 7	Experiment 5
8	Experiment 6
9	Experiment 7
10	Experiment 8
11	Experiment 9
12 to 13	Experiment 10 (mini-project)
14	Viva and model practical examination